

IT1244

Artificial Intelligence: Technology & Impact

Week 5 Tutorial 3

TODAY'S AGENDA

1. Clarifying a problem
2. Recap of concepts
3. Discussion of tutorial questions
4. Pollev Questions



CLARIFICATION

CLARIFYING QUESTION

Someone asked me a great question!!

Which of the following characteristics is true of all uninformed search algorithms? 00:45 18

- A) They always find the shortest path to the solution. 0%
- B) They rely on heuristics to guide the search process. 0%
- ✓ C) They do not have prior knowledge about the number of steps to the goal. 100%
- D) They only work for problems with a finite number of states. 0%

BFS or DFS or A-star search algorithm can only be used if the search space is finite?

✓ ☐ True

✗ ☒ False

Explanation

True, if the search space is infinite, then the search tree will not be expanded completely. These search algorithms works only in the finite state space settings.

They contradict! Which is correct?

CLARIFYING QUESTION

Ans from Prof: “They can be applied to infinite spaces, but they just don’t guarantee termination.”

Which of the following characteristics is true of all uninformed search algorithms? 00:45 18

- A) They always find the shortest path to the solution. 0%
- B) They rely on heuristics to guide the search process. 0%
- ✓ C) They do not have prior knowledge about the number of steps to the goal. 100%
- D) They only work for problems with a finite number of states. 0%

BFS or DFS or A-star search algorithm can only be used if the search space is finite?

✓ ☐ True

✗ ☒ False

Explanation

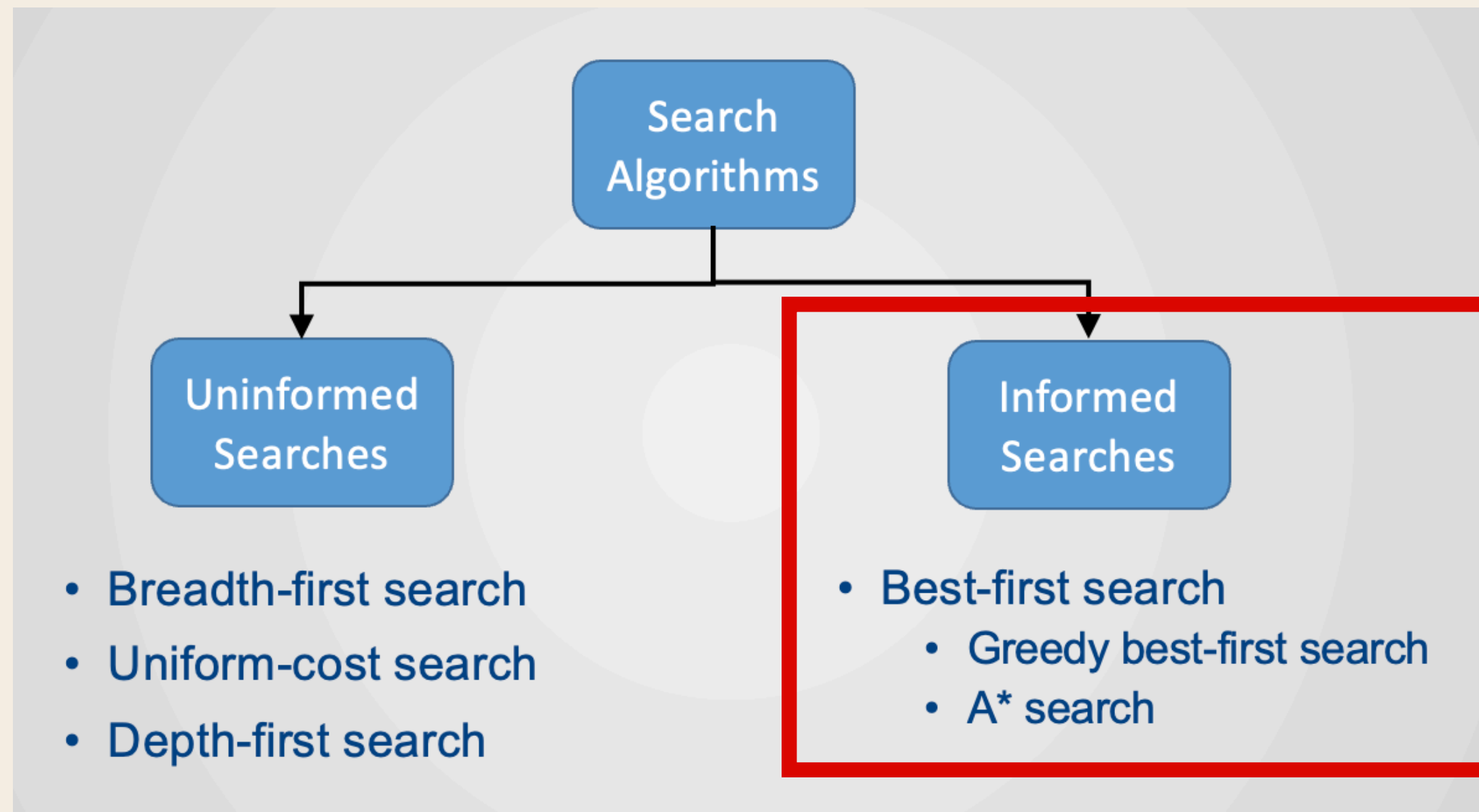
True, if the search space is infinite, then the search tree will not be expanded completely. These search algorithms works only in the finite state space settings.

Conclusion: Pollev correct, Lecture video quiz incorrect

RECAP

INFORMED SEARCH

- Use heuristic function $h(n)$ to **estimate** the **cost to the goal**
 - Can tell which non-goal states are better by comparing $h(n)$.



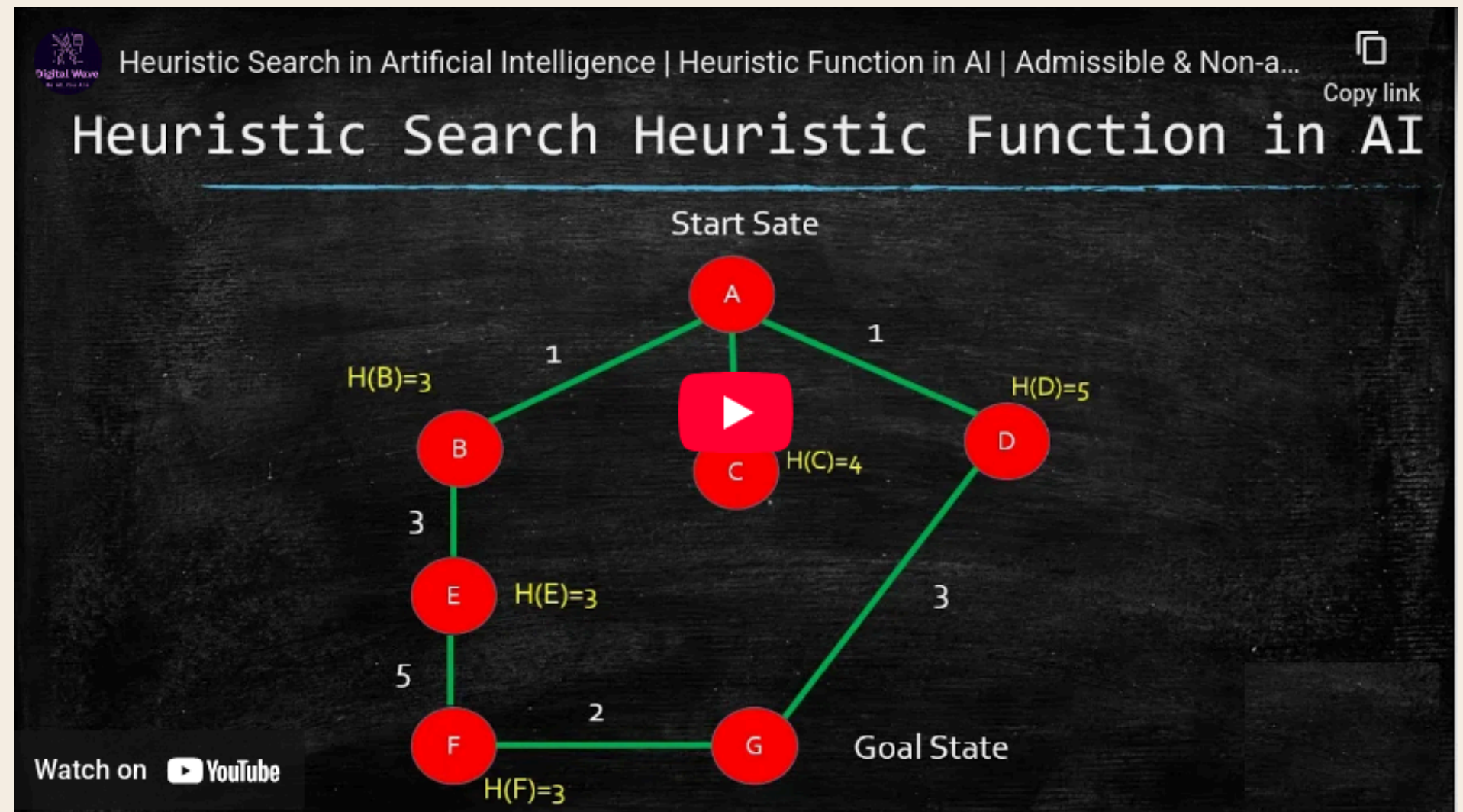
HEURISTIC FUNCTION

- A heuristic function estimates the cost or distance between a specific state and the goal $\rightarrow$ guiding algorithm to reach the goal with fewer steps

Two Properties for good

heuristic:

1. Admissable
2. Consistent



<https://www.youtube.com/watch?v=snRwKCK50Ug>

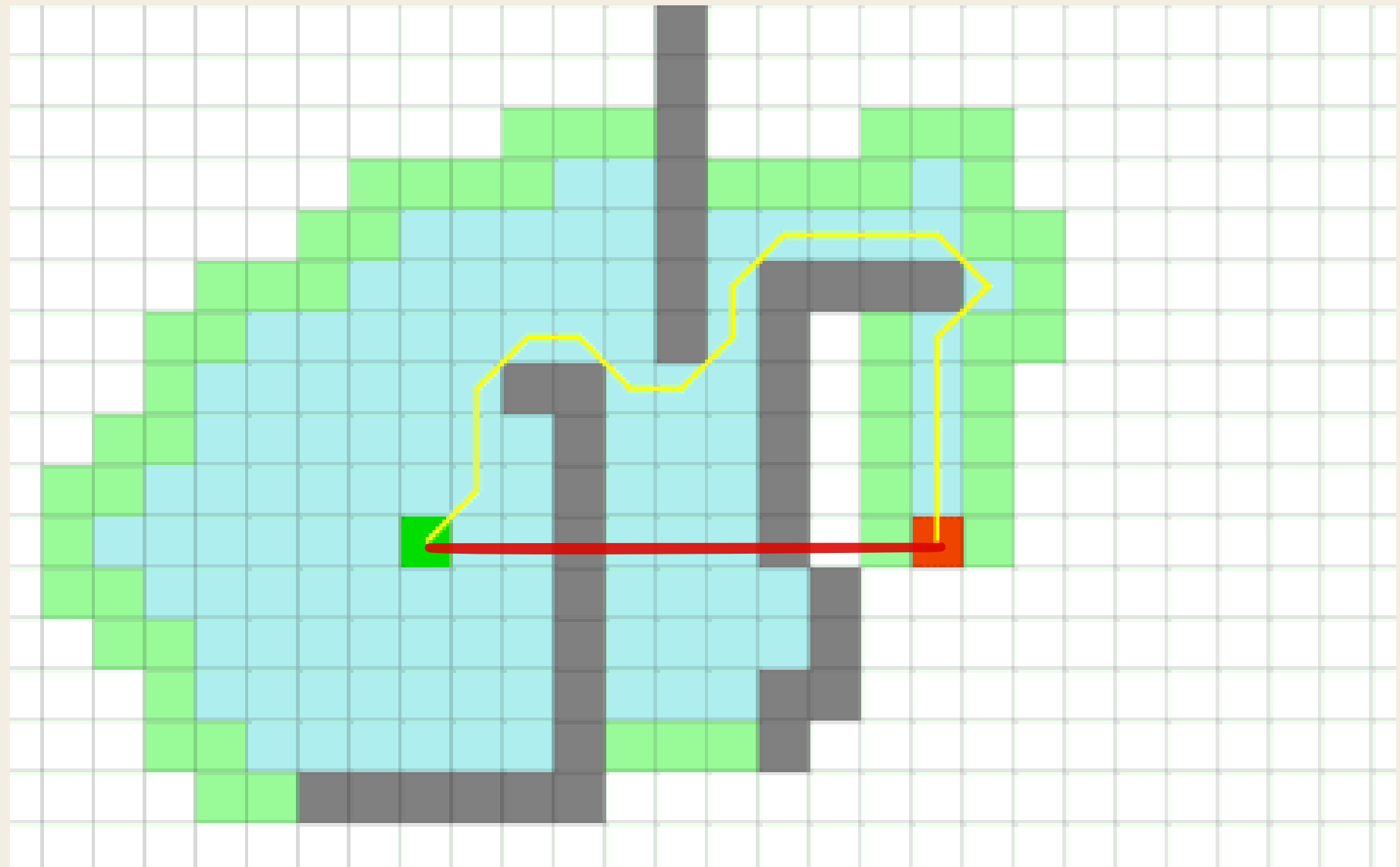
ADMISSABLE

- For every node, $h(n) \leq$ true cost to reach the goal from node n . In other words, **never overestimates** the path cost to the goal.

Example 1:

Manhattan Distance ignoring the obstacle.

True cost with obstacle will confirm have larger cost than that.



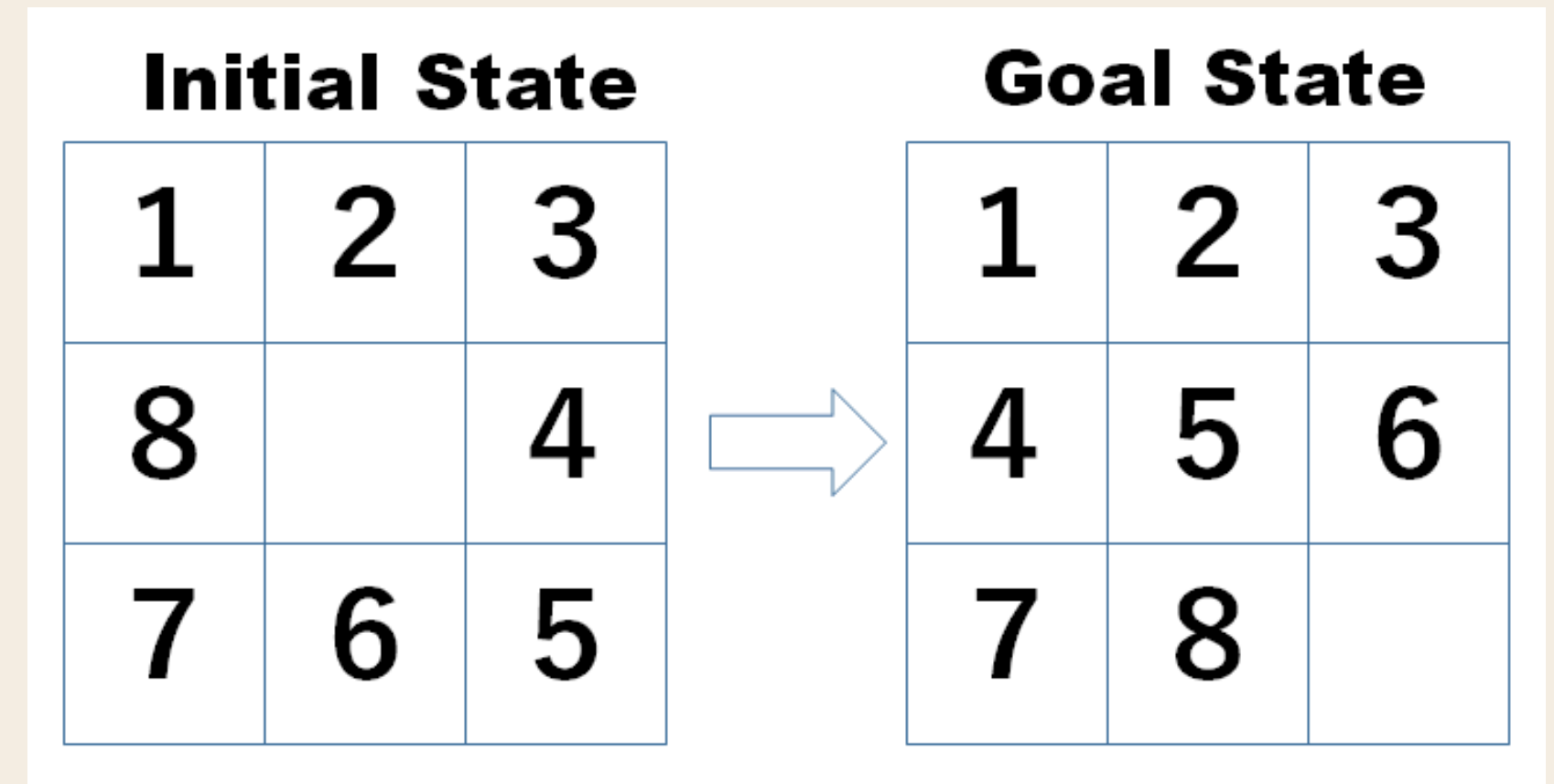
ADMISSABLE

- For every node, $h(n) \leq$ true cost to reach the goal from node n . In other words, **never overestimates** the path cost to the goal.

Example 2:

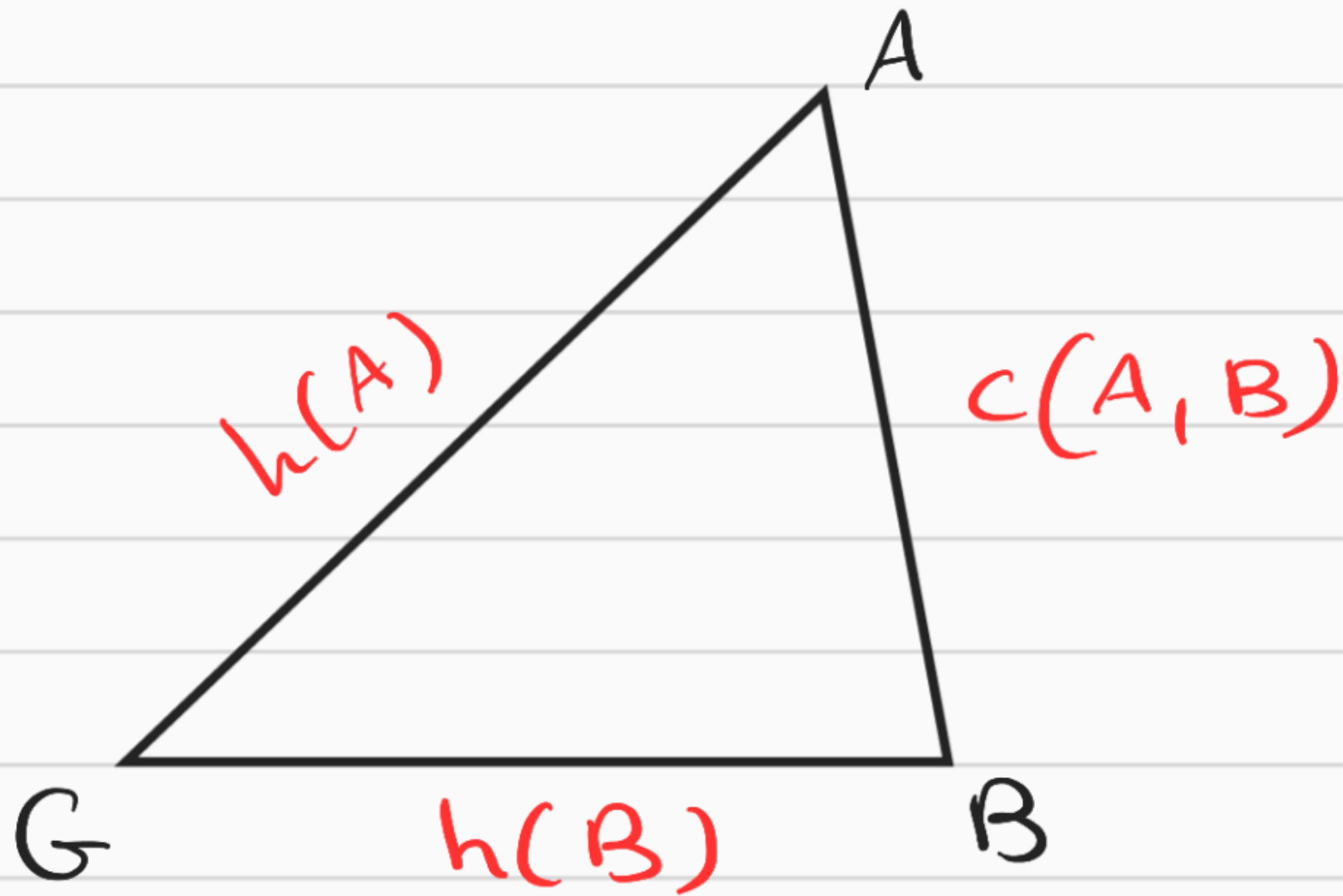
Let heuristic be number of misplaced tiles (eg. 4 misplaced tiles {8,4,6,5}).

So we know to get to the goal state, we **MINIMALLY** need to make at least 4 steps, though the true cost may be more than that.



CONSISTENT (MONOTONIC)

- For any node n , $h(n) \leq \text{actual cost}(n \text{ to } n') + h(n')$. In other words, the heuristics function adheres to triangle inequality ($c \leq a + b$)



****Consistency** means your estimate to the goal cannot drop by more than the cost of one step when you move one step closer.

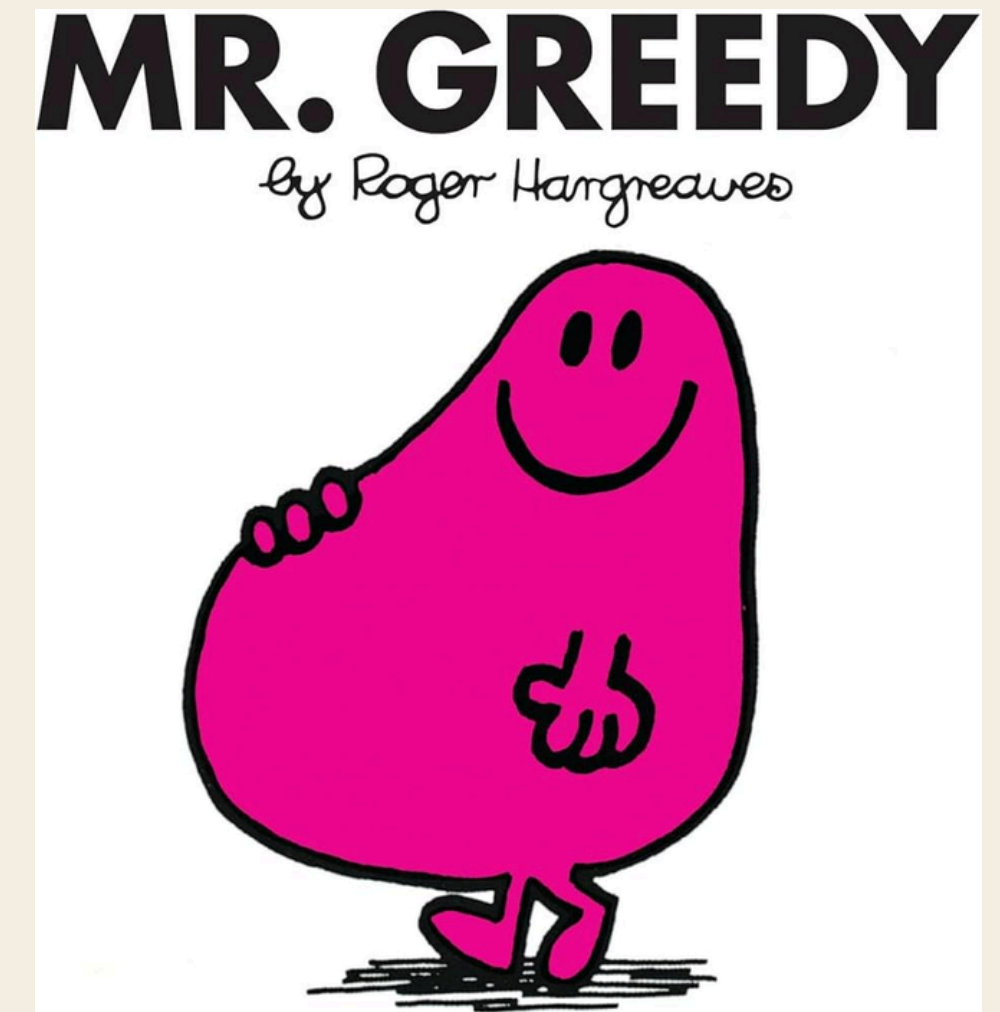
GREEDY BEST FIRST SEARCH

Characteristics

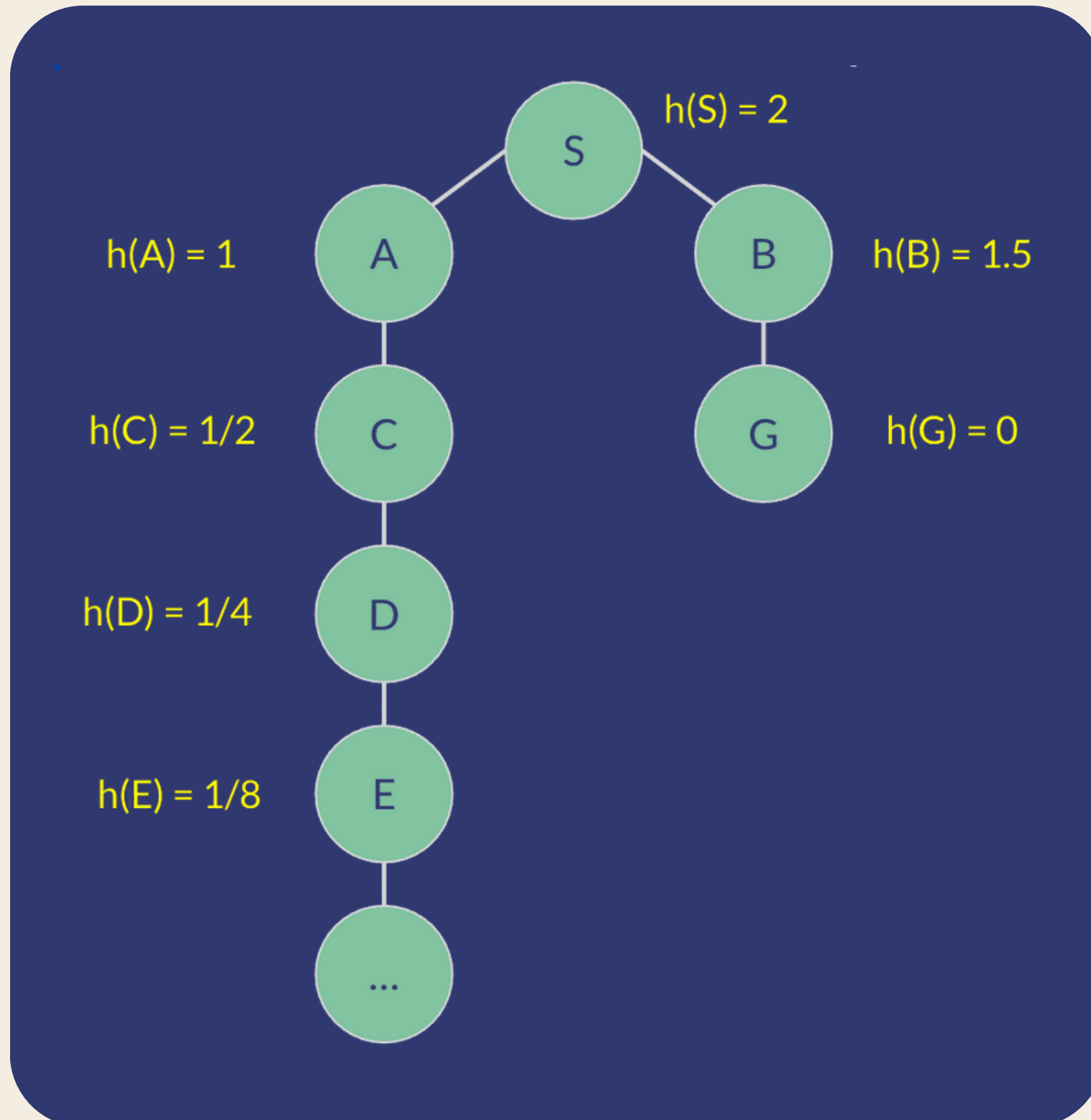
- Expands node that “looks” closest to the goal (using $h(n)$ only)
- Makes greedy, local decisions $\rightarrow$ ignores cost from start node $g(n)$
- Frontier: Priority Queue ordered by $h(n)$

Properties

- **Complete:** Yes (if state space is finite)
- **Optimal:** No (only consider local optimum; discard other possible path from previous nodes)
- **Time & Space Complexity:** Exponential if bad heuristics is used



GREEDY BEST FIRST SEARCH



- GBFS keeps expanding the left branch because each node has a slightly smaller heuristic ($h = 2^{-i}$), though there's a more optimal path to the goal
- Here we can see the consequence of ignoring path cost $g(n)$ (cost from the start node)

Credit to previous TA Jason for this

A* SEARCH

Characteristics

- Improvement from GBFS by integrating actual path cost, $g(n)$, in the evaluation function, $f(n)$

$$f(n) = g(n) + h(n)$$

- [Total est. cost = Actual cost so far + Estimated cost to goal]
- Frontier: Priority Queue

Properties

- **Complete:** Yes, if there are finite no of nodes
- **Optimal:** Yes, if $h(n)$ is consistent
- **Time & Space Complexity:** Bad (stores all generated nodes in memory, run out of space before time)

This proof shows how A* always expands the node with smallest $f(n)$

$$h(n) \leq 1 + h(n')$$

$$g(n) + h(n) \leq g(n) + 1 + h(n') = g(n') + h(n')$$

$$f(n) \leq f(n')$$

Optimal: u can't reach a node later with a smaller $f(n)$ than its parent

A* SEARCH

Illustration:

Step 0: Add a into Frontier $\rightarrow$ PQ = {(a, 14)}

Step 1: Expand a $\rightarrow$ PQ = {(c, 14), (b, 16)}

Step 2: Expand c $\rightarrow$ PQ = {(b, 16), (d, 16), (e, 17)}

Step 3: Expand b $\rightarrow$ PQ = {(d, 16), (e, 17), (f, 20)}

*** don't update worst path. $a \rightarrow b \rightarrow e = 20 > 17$

Step 4: Expand d $\rightarrow$ PQ = {(e, 16), (f, 20)}

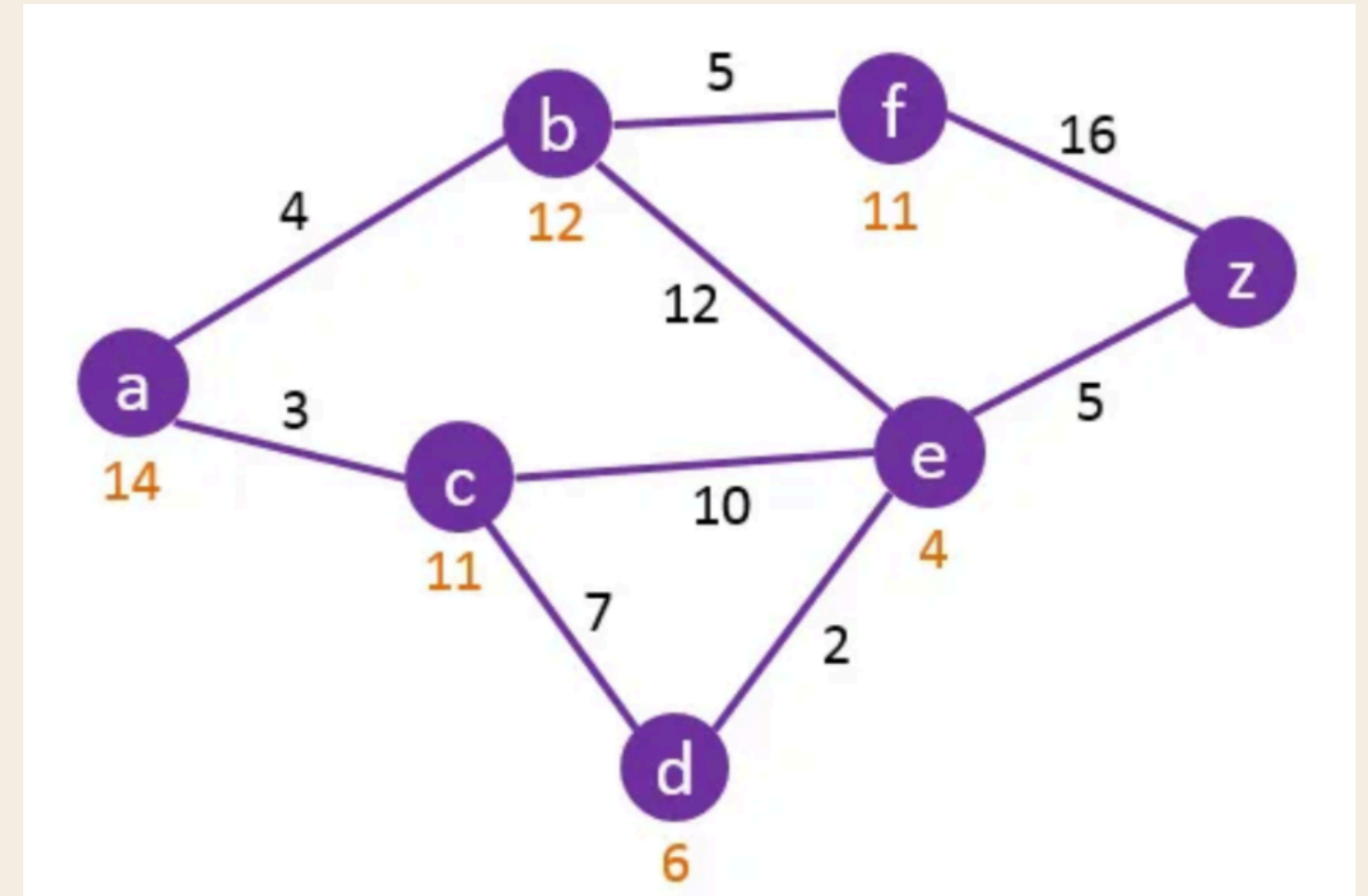
*** update frontier since better path is found

Step 5: Expand e $\rightarrow$ PQ: {(z, 17), (f, 20)}

*** don't revisit expanded nodes (node D)

Step 6: Expand z $\rightarrow$ PQ: {(f, 20)}

*** Goal reach; Optimal path found: $a \rightarrow c \rightarrow d \rightarrow e \rightarrow z$; PC = 17



Heuristic value $h(n)$;
Actual Path Cost $g(n)$;

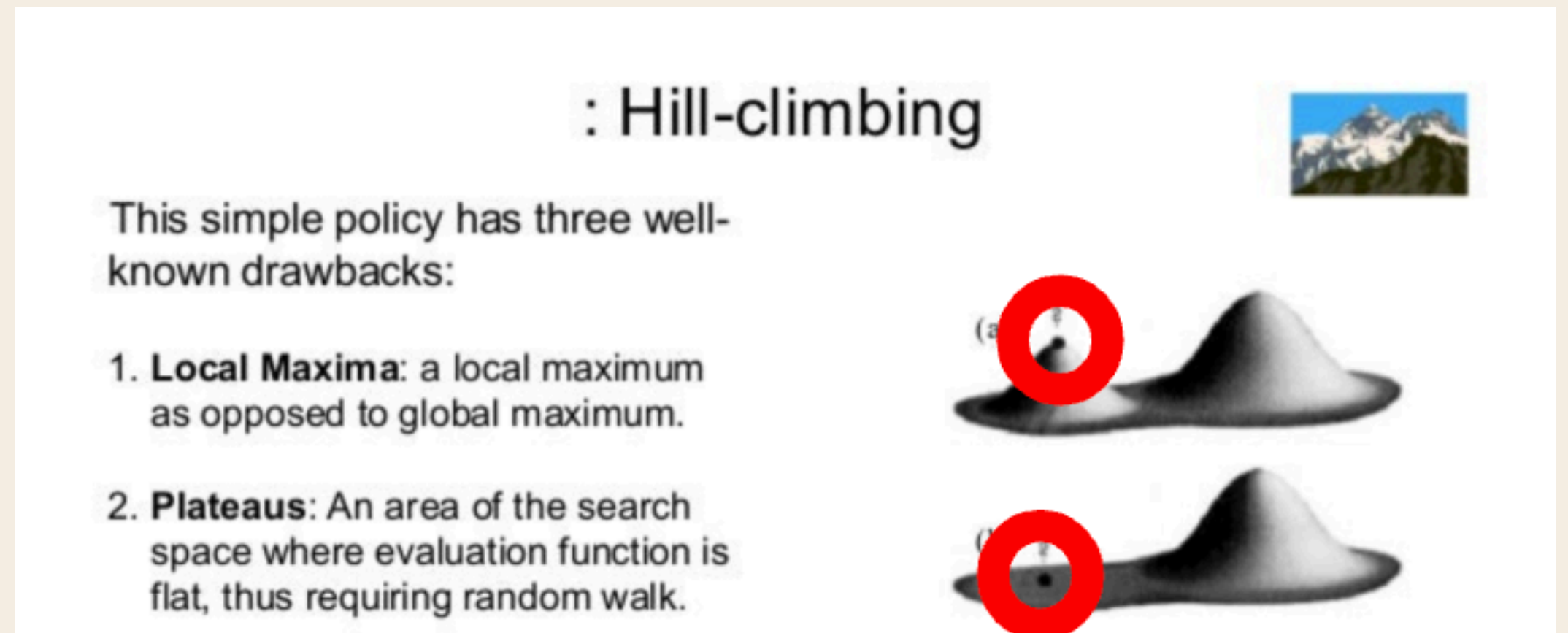
LOCAL SEARCH: HILL CLIMBING

Some Limitations:

- Getting stuck on local optima
- Flat ground (plateau)
- Wrong starting point

Solution(s):

1. Randomise starting point
2. Run the algorithms many times and take the best result

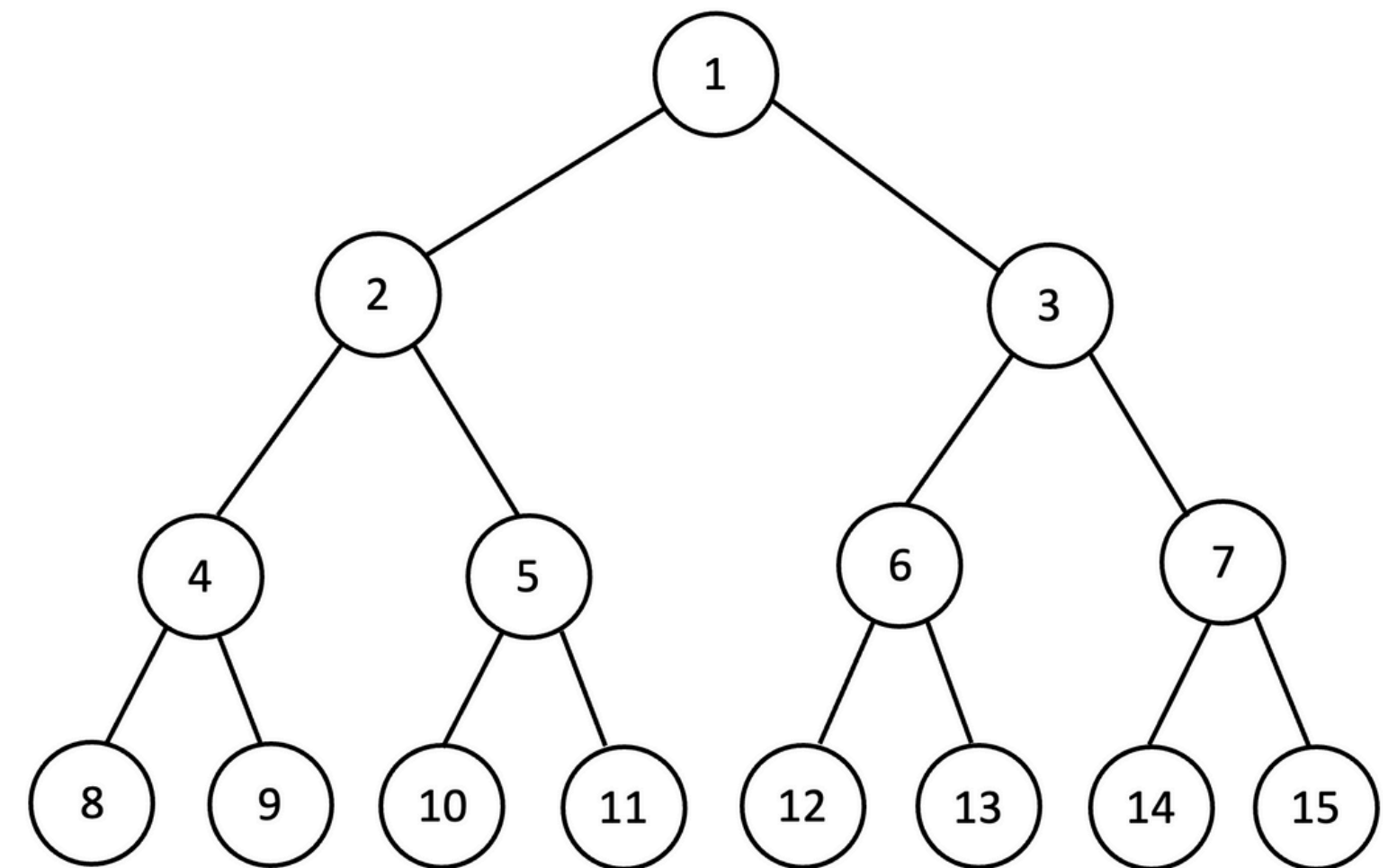


TUTORIAL QUESTIONS

TUTORIAL QUESTION 1

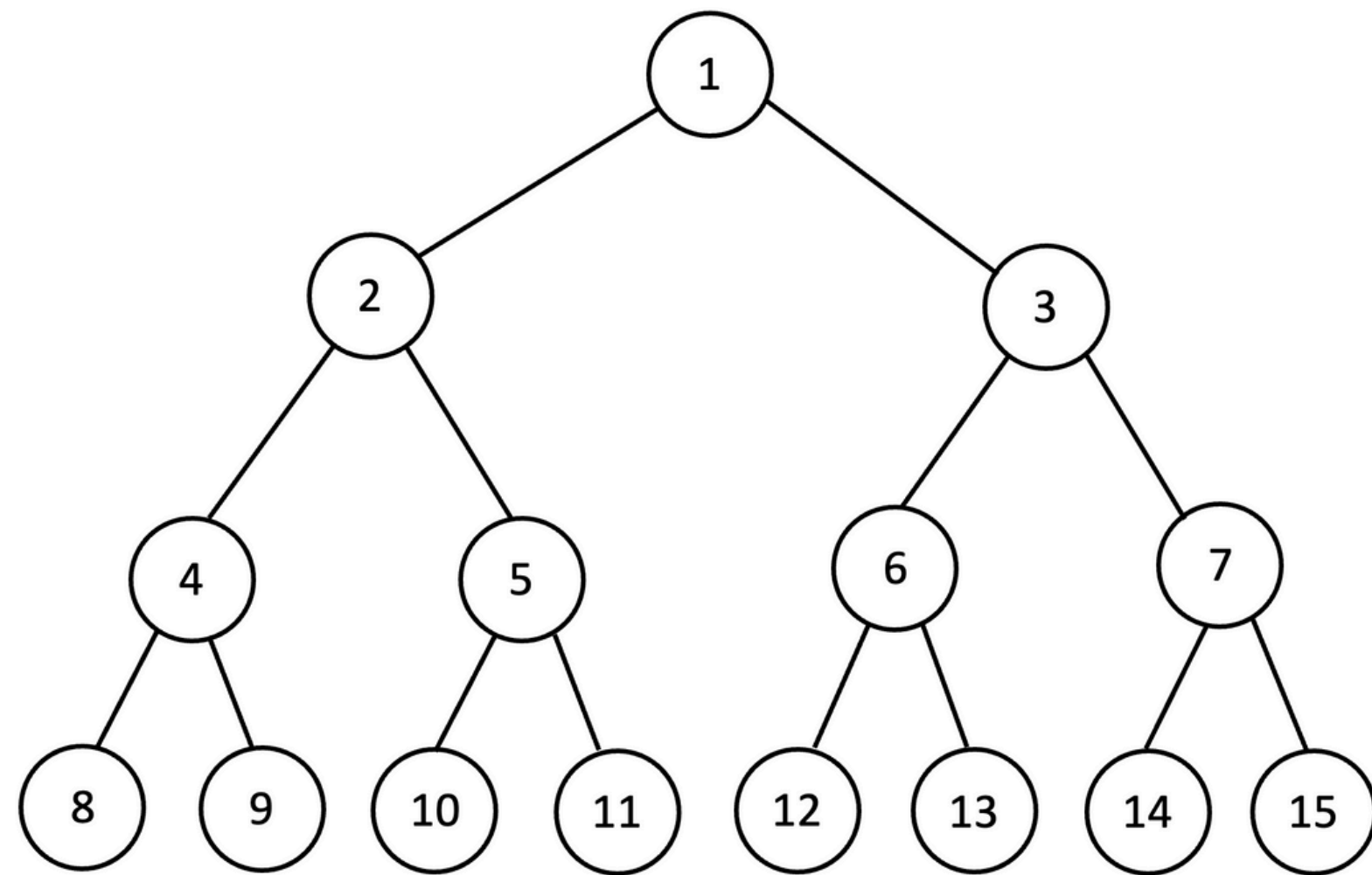
Consider a search tree with the start state as number 1. Each state k has two successors/child nodes: numbers $2k$ and $2k + 1$.

a. Draw the portion of the search tree for states 1 to 15. You need to start with the start state of $k=1$.



TUTORIAL QUESTION 1

b. Suppose the goal state is 11. List the order in which nodes will be visited for BFS and DFS.

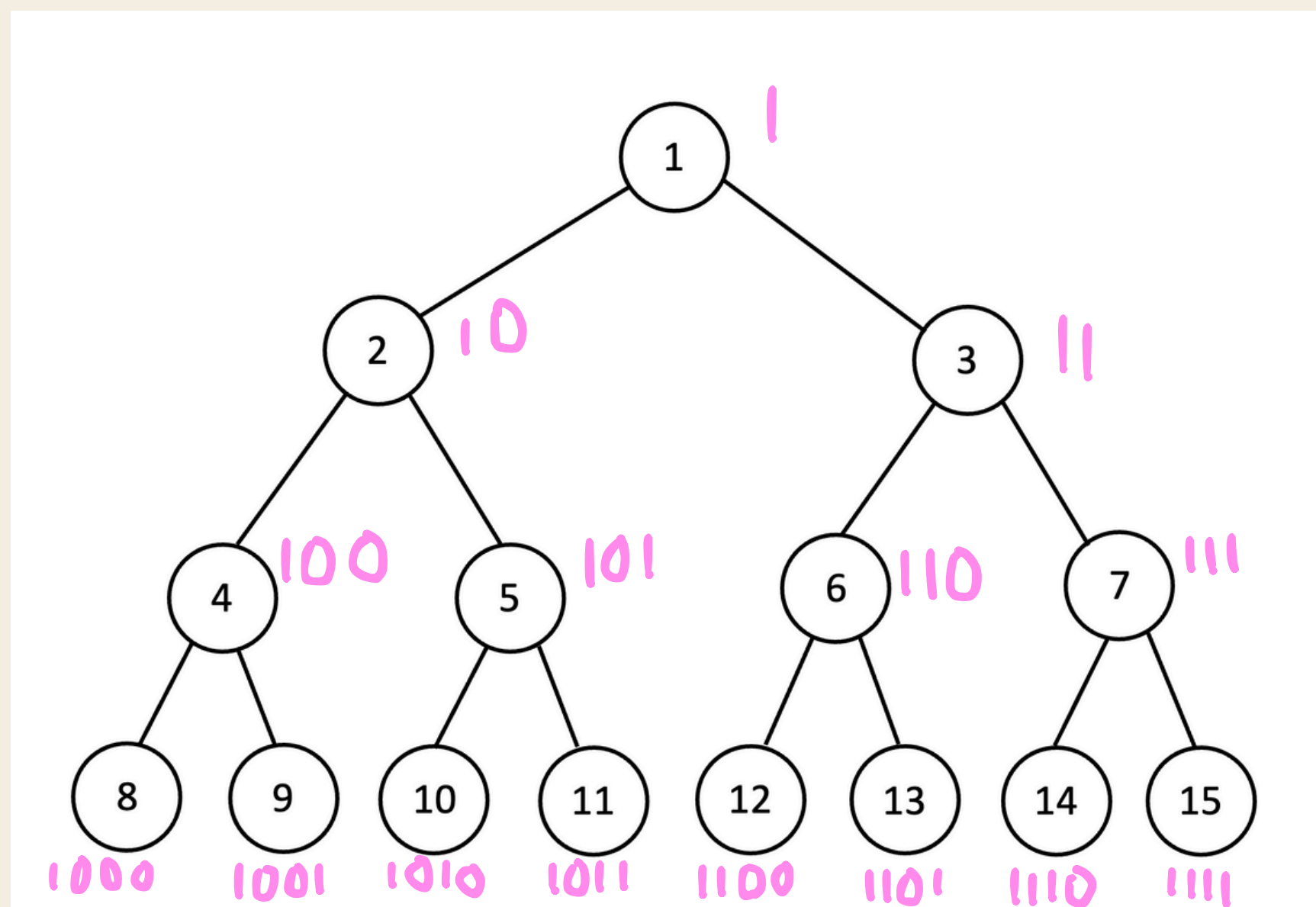


BFS: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11

DFS: 1, 2, 4, 8, 9, 5, 10, 11

TUTORIAL QUESTION 1

c. Can you find an algorithm that outputs the solution to this problem without any search at all?

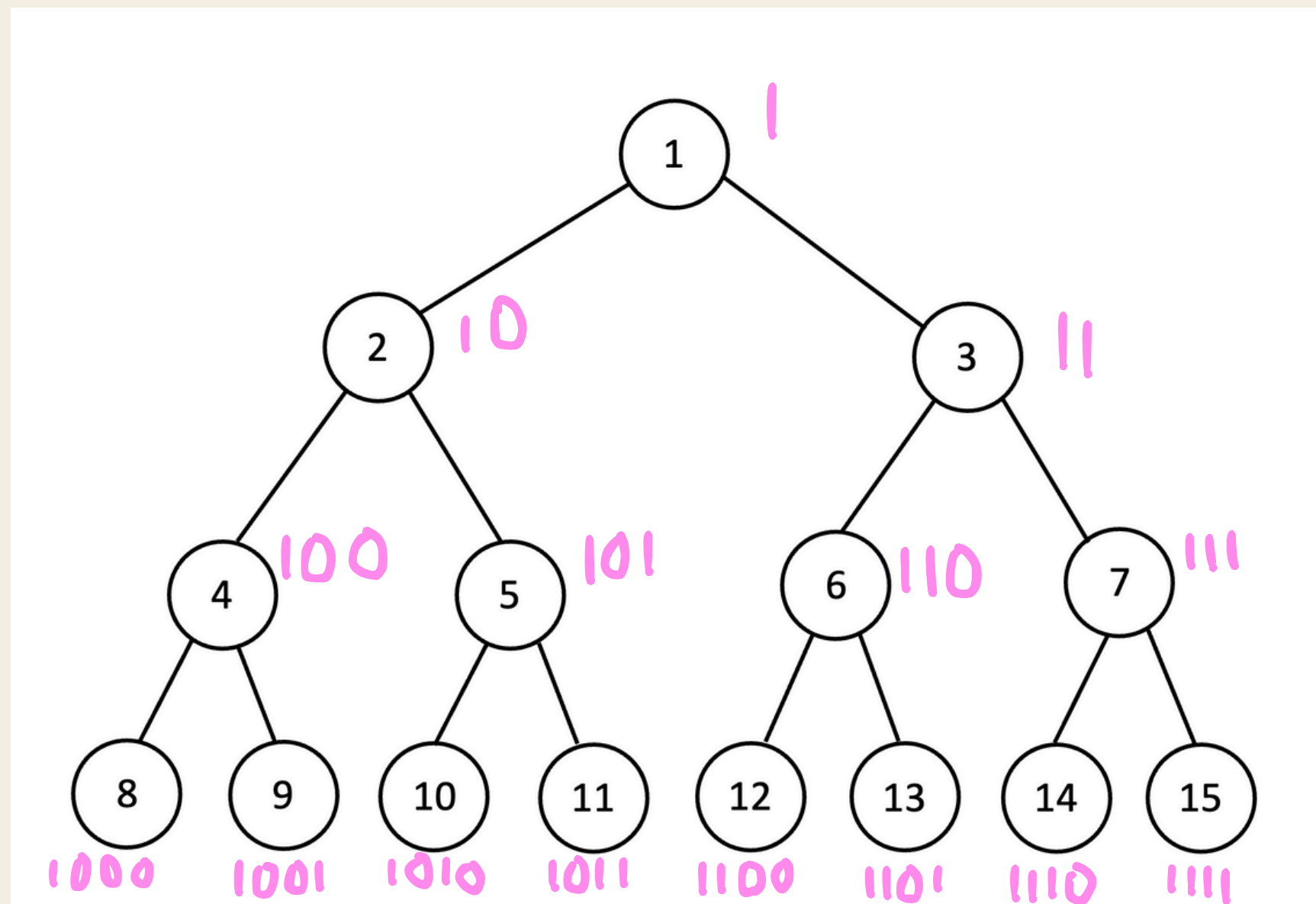


Use binary!

- Ignore the first digit (left-most)
- For each digit, if it is '0' then move left, otherwise move right

TUTORIAL QUESTION 1

c. Can you find an algorithm that outputs the solution to this problem without any search at all?



```
def find_node(n):  
    binary = bin(n)[2:] # Get binary: bin(5) = '0b101' -> '101'  
    path = binary[1:]   # Skip first bit: '101' -> '01'  
    print(path)  
  
    result = ""  
    for bit in path:  
        if bit == '0':  
            result += 'L'  
        else:  
            result += 'R'  
  
    return result  
  
# Test  
find_node(11) # 'LRR'  
find_node(5) # 'LR'
```

TUTORIAL QUESTION 2

(i) [3x3 N-puzzle Problem] How many number of possible states are there?

9! ($9 \times 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1$)

For first tile, there is 9 possible location, once we fix the first tile, then there are 8 possible locations for second tile, once we fix the second tile, there will be 7 possible locations for third tile, and so on.

1	2	5
3	4	
6	7	8

TUTORIAL QUESTION 2

(i) [3x3 N-puzzle Problem] How many number of possible states are there?

But only $9!/2$ solvable.

So, the reachable number of states for 3x3 puzzle is $9!/2$.

Johnson & Story (1879) used a **parity** argument to show that half of the starting positions for the n puzzle are impossible to resolve, no matter how many moves are made. This is done by considering a binary function of the tile configuration that is **invariant** under any valid move and then using this to partition the space of all possible labelled states into two mutually inaccessible **equivalence classes** of the same size. This means that half of all positions are unsolvable, although it says nothing about the remaining half.

https://en.wikipedia.org/wiki/15_puzzle

1	8	2
	4	3
7	6	5

Given State
Solvable

We can reach goal state by sliding tiles using blank space.

8	1	2
	4	3
7	6	5

Given State
Not Solvable

We can not reach goal state by sliding tiles using blank space.

TUTORIAL QUESTION 2

(ii) Explain the issues if BFS or DFS search algorithms are used for this problem.

Since there are large number of states, using **BFS may run out-of-memory** and using **DFS may return sub-optimal solution**

Initial state:

1	2	3
-	4	8
7	6	5

Goal state:

1	2	3
4	5	6
7	8	-

Let's try together: <https://tristanpenman.com/demos/n-puzzle/>

TUTORIAL QUESTION 2

(iii) If you are using A-Star search, then what will be your heuristic function.

Manhattan distance:

- sum of the number of moves required for each tile to reach its correct position in the goal state.

$H(n) = 2 + 1 + 0 + 1 + 2 + 2 + 1 + 2 = 11$
respectively for the tiles with the
numbers 2, 5, 3, 1, 7, 4, 8, and 6.

Current State

2		5
3	1	7
4	8	6

Goal State

	1	2
3	4	5
6	7	8

Let's try together: <https://tristanpenman.com/demos/n-puzzle/>

TUTORIAL QUESTION 2

(iii) If you are using A-Star search, then what will be your heuristic function.

Hamming Distance:

number of tiles that are misplaced from their respective location in the final goal state.

$H(n) = 1 + 1 + 0 + 1 + 1 + 1 + 1 + 1 = 7$
respectively for the tiles with numbers
2, 5, 3, 1, 7, 4, 8, and 6.

Current State

2		5
3	1	7
4	8	6

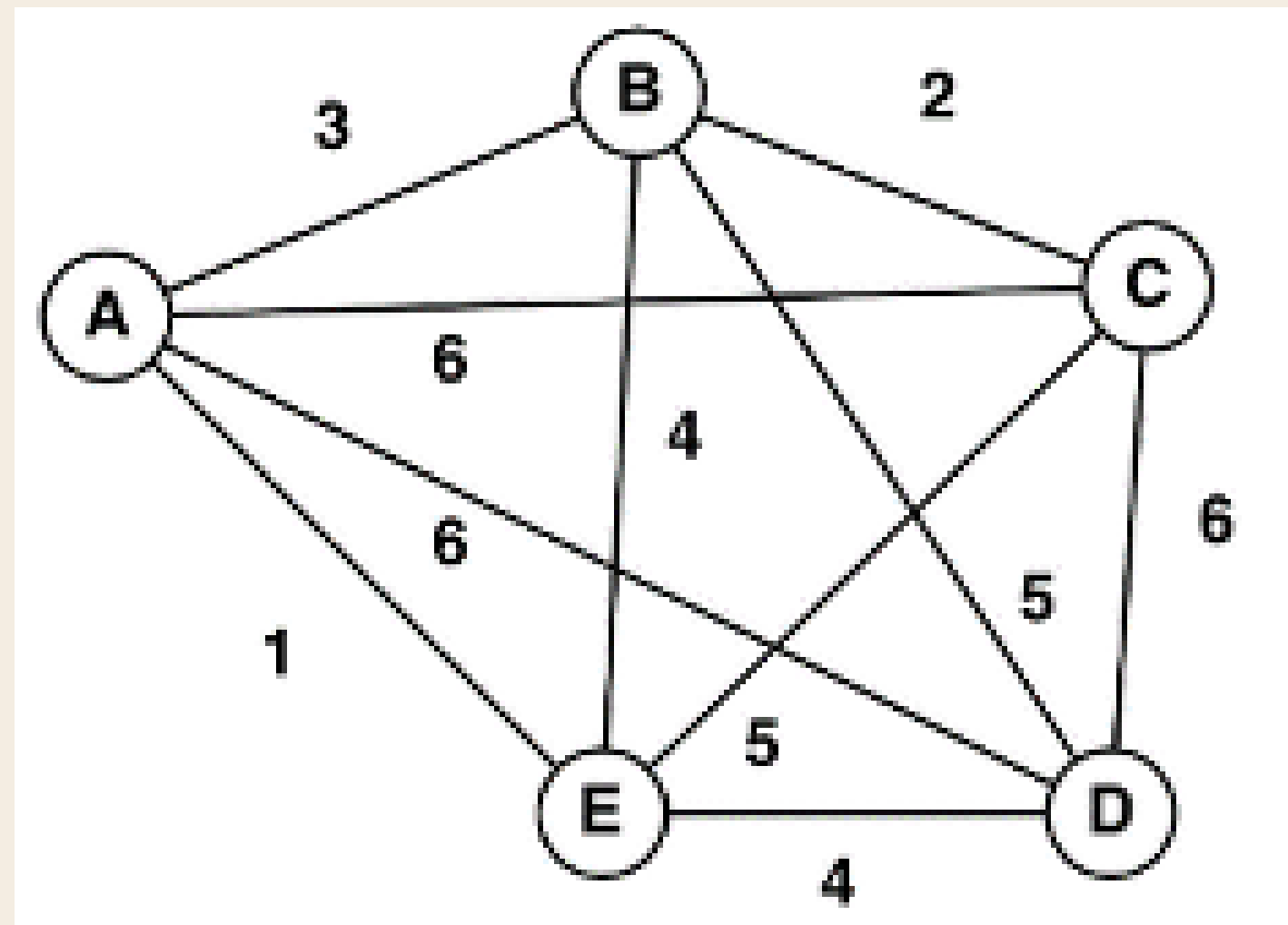
Goal State

	1	2
3	4	5
6	7	8

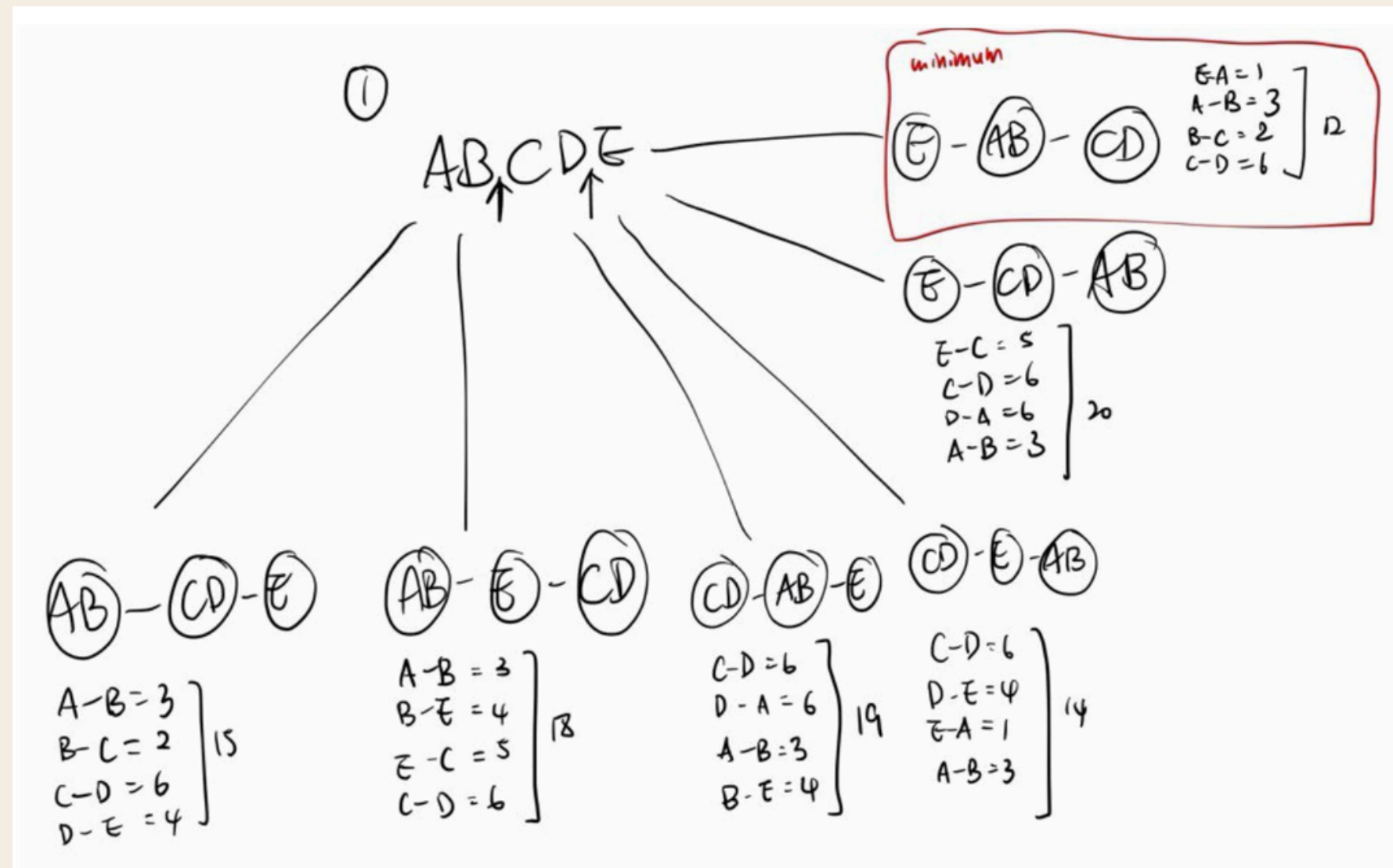
Let's try together: <https://tristanpenman.com/demos/n-puzzle/>

TUTORIAL QUESTION 3

There are 5 cities in a map and the salesperson need to travel all these 5 cities once such that the total distance travelled is minimum. Explain how would you solve this problem using simple **hill-climbing search algorithm**.



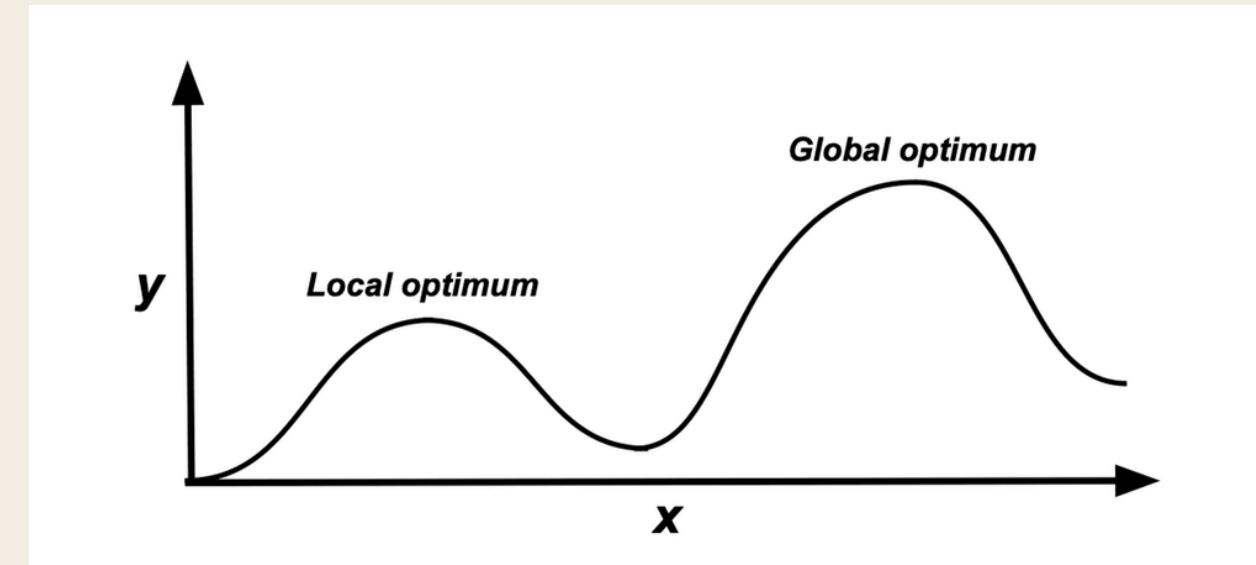
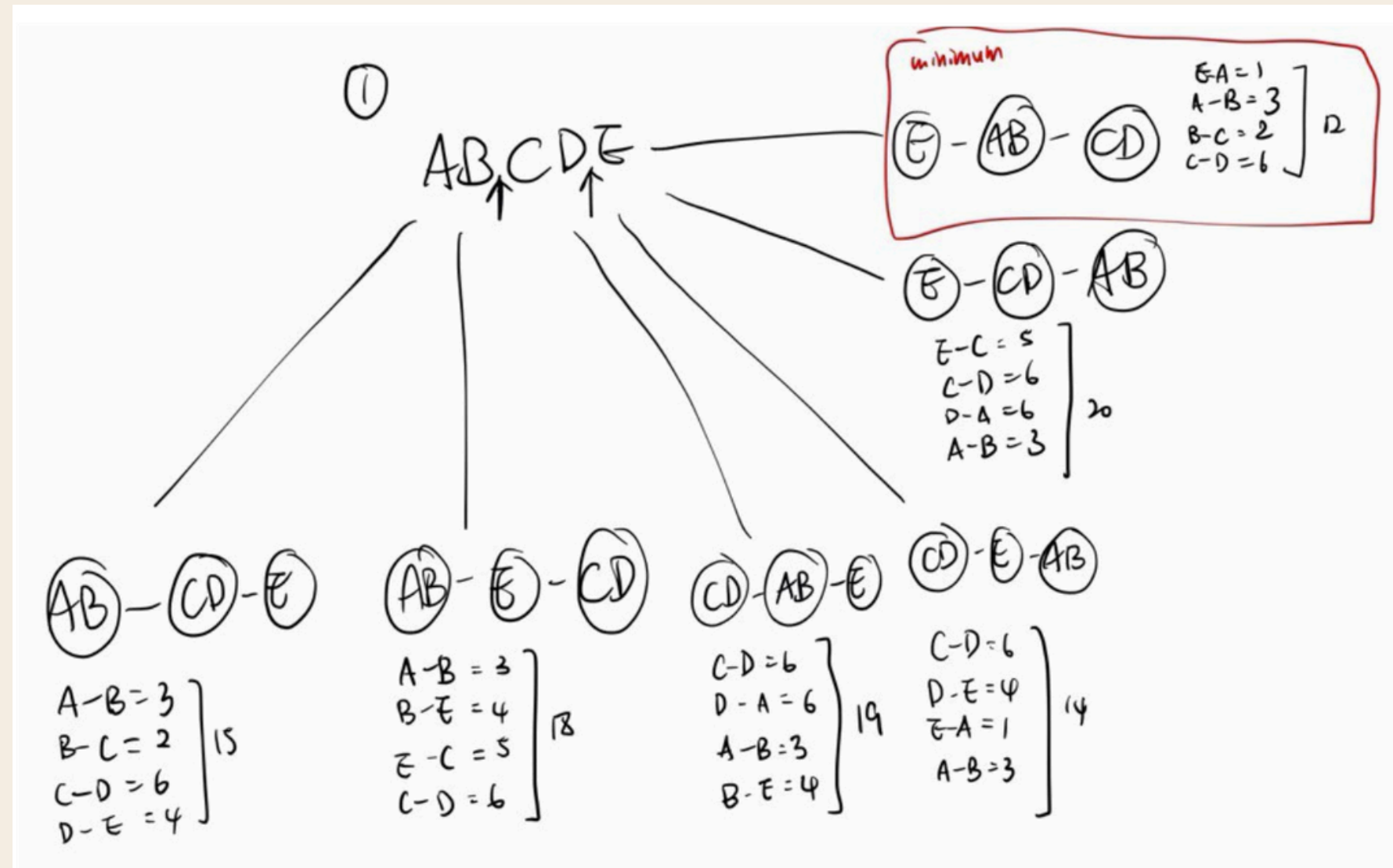
TUTORIAL QUESTION 3



1. Connect all the cities into an arbitrary path.
2. Pick two points along the path at random.
3. Split the path at those points, producing three pieces.
4. Try all six possible ways to connect the three pieces.
5. Keep the best one, and reconnect the path.
6. Repeat until no improvement is observed for a while.

Code Demo on Github

TUTORIAL QUESTION 3



Hill-climbing

- Only update if improved
- Starts at random point
- Finds local solution (not guaranteed to be best)
- Repeats until no improvement is found

Code Demo on Github

TUTORIAL QUESTION 4

Q4. Give two examples of optimization problems in your field of study and explain how would you apply hill-climbing search for it.

Data Science:

- **Feature selection:** Start with a random set of features, iteratively add/remove individual features, and only keep changes that improve model accuracy until no single feature change helps.
- **Hyperparameter Tuning for Deep Learning:** Start with random hyperparameter values, make small adjustments to one parameter at a time, and only keep changes that reduce validation loss until no adjustment improves performance.

QUESTION OF THE WEEK

A consistent heuristic functions must be admissible but not the other way around? Why?

QUESTION OF THE WEEK

A consistent heuristic functions must be admissible but
not the other way around? Why?

Consistent:

$$\begin{aligned} h(N_{k+1}) &\leq c(N_{k+1}, N_k) + h(N_k) \leq c(N_{k+1}, N_k) + c(N_k, N_{k-1}) + h(N_{k-1}) \leq \dots \\ &\leq c(N_{k+1}, N_k) + c(N_k, N_{k-1}) + c(N_{k-1}, N_{k-2}) + \dots + c(N_1, N_0) + h(N_0) \end{aligned}$$

true cost

Goal Node:
 $h(N_0) = 0$

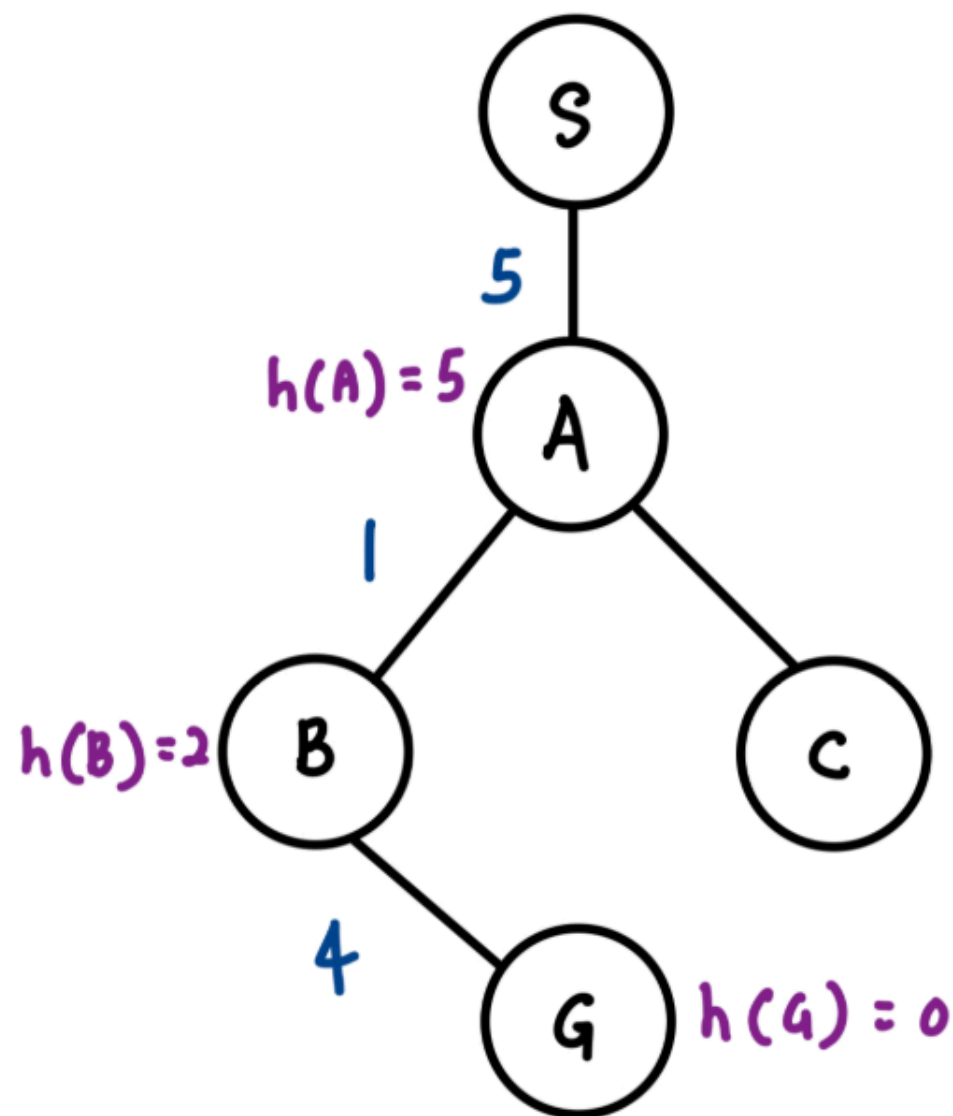
$$h(N_{k+1}) \leq \text{true cost}$$

Admissible!

QUESTION OF THE WEEK

A consistent heuristic functions must be admissible but
not the other way around? Why?

Counterexample: Admissible does not imply consistent!



Check that both $h(A)$ and $h(B)$ are admissible:

- The actual path cost from A to G is $1 + 4 = 5 = h(A)$ ✓
- The actual path cost from B to G is 4 and $h(B) = 2 < 4$ ✓

Check consistent:

$$h(A) \leq 1 + h(B) \Leftrightarrow 5 \leq 1 + 2 = 3$$

$5 \leq 3$ is a contradiction! ✗

POLLEV QUESTIONS

Join by Web:

PollEv.com/kaironglee

Join by QR:



NEXT WEEK

Machine Learning & KNN

Linear Model Nearest Neighbor



NOOOOOOOOOO!

You can't just pick the closest datapoint and classify it as that!!

You need to prove $E_{in} = E_{out}$ and use Linear Algebra and use a feature transform and cross validate and calculate bias and variance and



This looks like something I've seen before. It's probably the same thing